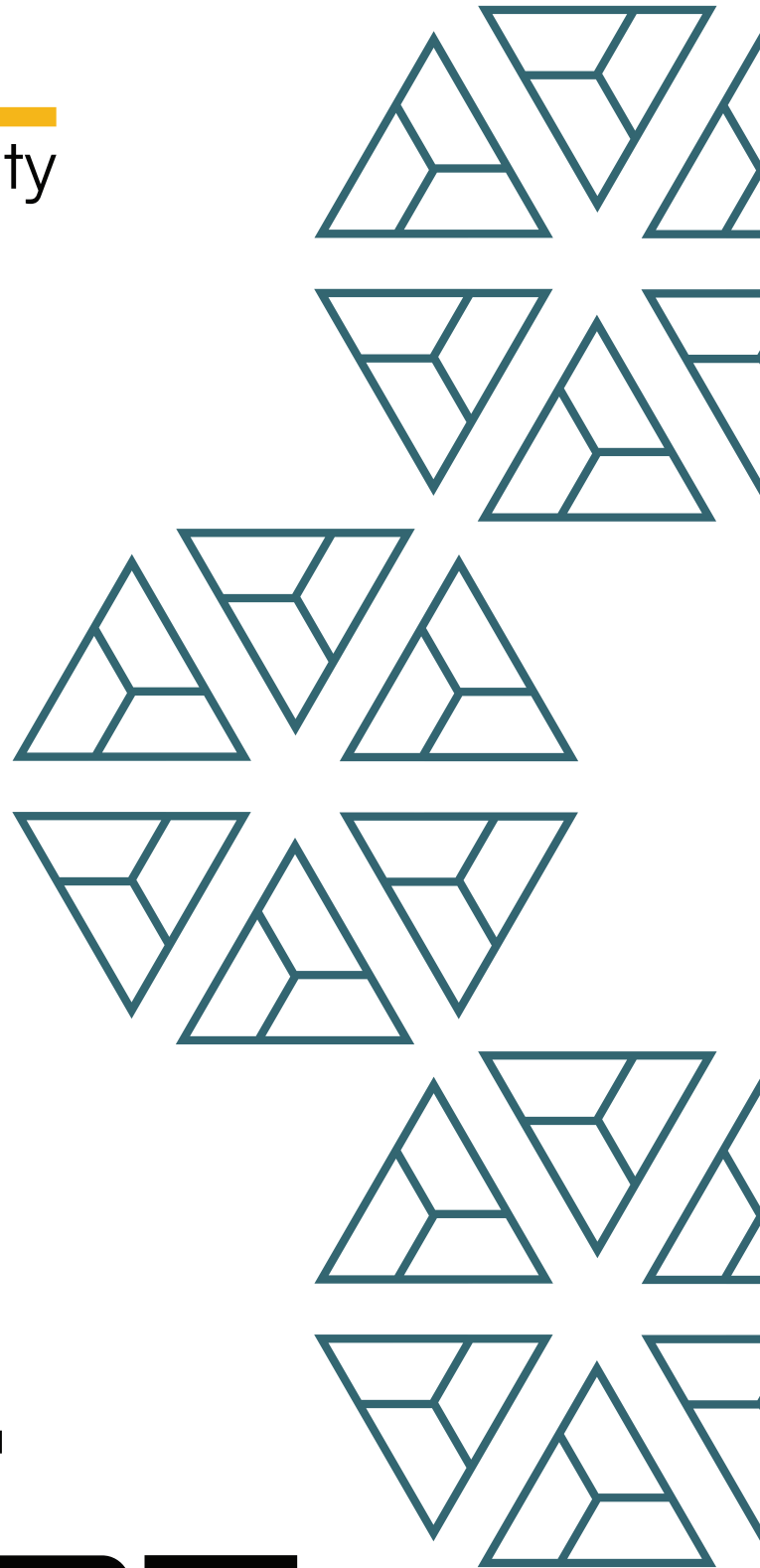




BAIL
security



Lista Dao
RWA

FINAL REPORT

October '2025

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista DAO - RWA
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/lista-new-contracts/tree/78a3aff32bd13da3089dcadb9bd94f9458285
Resolution 1	https://github.com/lista-dao/lista-new-contracts/tree/b86a34731ea06b88bb426ad172b85e384a282bc6

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High	2	1	1			
Medium	2	2				
Low	4	1		3		
Informational	10	5		5		
Governance						
Total	18	9	1	8		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

RWAEarnPool

The [RWAEarnPool](#) contract allows users to stake `usd1` and gain access yield that is generated by interest paid from the [RWAAdapter](#). Each time the [RWAAdapter](#) accrues interest, it notifies the Earn Pool via [notifyInterest](#). The amount is then dispersed as yield over the course of 1 week.

Users cannot simply withdraw their assets, users must first queue a request to withdraw their assets. This is due to the asynchronous nature of the vault that [RWAAdapter](#) deposits to. A bot is in charge of calling [finishEarnPoolWithdraw](#) which finalizes withdrawals as well as sends tokens to the [RWAEarnPool](#) contract. Once the withdrawal is finalized, users can receive the underlying asset by calling [claimWithdraw](#).

The contract also features whitelisting functionality which if activated, only allows deposits from users in the whitelist.

Appendix: Withdraw flow

The deposit flow is standard and not very complicated, deposit assets and get minted shares. The slight change can be seen in the withdraw flow which can be outlined as follows:

1. Users call [requestWithdraw](#), their shares are burned and asset amounts are queued in the [userWithdrawalRequests](#)
2. [RWAAdapter](#) contract must call [finishEarnPoolWithdraw](#) in order to confirm withdrawal requests which have been queued
3. Users can now call [claimWithdraw](#) in order to receive their assets

Core Invariants:

INV 1: If whitelist length $\neq 0$, users must be whitelisted to deposit

INV 2: Rewards are distributed throughout 1 week periods also known as `REWARD_DURATION`

INV 3: Batches can receive request for a max of 1 day, after which a new batch is formed

Privileged Functions

- `finishWithdraw`
- `notifyInterest`
- `setWithdrawFeeRate`

- setFeeReceiver
- setWhitelist
- emergencyWithdraw
- pause
- unpause
- grantRole
- revokeRole
- upgradeTo
- upgradeToAndCall

Issue_01	Withdrawals could be DoSed due to <code>userTotalAssets</code> not being updated properly
Severity	High
Description	Consider the following scenario → 1.) UserA and UserB are in the vault with 50 shares each i.e. <code>userTotalAssets</code> = 100 (1:1 share to asset ratio), and <code>totalAssets</code> is 500 due to profits and rewards accounted through <code>notifyInterest()</code> and assuming reward duration elapsed. 2.) UserA calls <code>requestsRedeem()</code> for his 50 shares -> amount comes out to be $50 \times 500 / 100 = 250$, but the tx would revert due to the operation <code>userTotalAssets -= amount;</code> Due to this requesting a withdrawal would be impossible unless further interest is notified through <code>notifyInterest()</code> which also updates <code>userTotalAssets</code> appropriately but even after this the contract can end up in the same problematic situation as the rewards get unvested.
Recommendations	<code>userTotalAssets</code> must be updated before <code>requestRedeem</code> execution, this can be done with a modifier which updates <code>userTotalAssets</code> .
Comments / Resolution	Fixed by following recommendations.

Issue_02	Attacker can steal assets for free due to share calculation being rounded down
Severity	High
Description	When requesting a withdraw an attacker can provide 1 wei as amount, the shares calculated would be <code>shares = convertToShares(amount);</code>, considering even the tiniest profit the share amount round to 0 and no share transfer to the fee receiver will take place and 0 shares would be burnt but the amount stored in the <code>userWithdrawalRequests[]</code> would be 1. This way an attacker can claim assets without burning any shares.
Recommendations	Make sure the share calculation is rounded up.
Comments / Resolution	Partially resolved. However please note that now there might be a slight discrepancy on the amount of assets withdrawn due to the new rounding implementation, as now <code>shares</code> is rounded up during withdrawals but the value of <code>assets</code> is still rounded down during deposits. The fix however, mitigates free asset withdrawal which was the main issue.

Issue_03	Fee could never be 100% claimed
Severity	Medium
Description	When requesting a withdraw with <code>requestWithdraw()</code> a percentage of the user's shares are transferred to the <code>feeReceiver</code>, for example - If Alice requested a withdrawal for 100 shares then 90 shares worth of assets are accounted in the <code>userWithdrawalRequests[Alice]</code> and 10 shares are transferred to the fee receiver. This means that the fee receiver should be able to claim assets worth 10 shares, but the only path to redeem shares is through <code>requestWithdraw()</code> where if the fee receiver requests 10 shares to withdraw a 10% fee is again applied. This means that the fee receiver can never claim 100% of its fee shares.
Recommendations	Fee receiver should not pay a fee on his withdrawal.
Comments / Resolution	Fixed by following recommendations.

Issue_04	rewards per day may be decreased when <code>notifyInterest</code> is called with a low value
Severity	Low
Description	In order to understand the issue, consider the following scenario: 1. 25 wei is unvested and is to be distributed among 5 days, this assumes 5 wei per day 2. <code>notifyInterest</code> is called with 1 wei 3. The <code>periodStart</code> is reset 4. now the rewards are spread apart 7 days again 5. we are now giving out 26 wei in rewards among 7 days which is 3.7 wei 6. users who were only going to continue staking for 3 days will have incurred less rewards compared to if <code>notifyInterest</code> was never called
Recommendations	Consider this scenario when <code>notifyInterest</code> is to be called with a low value.
Comments / Resolution	Acknowledged.

Issue_05	Fee amount is rounded down in favour of the users
Severity	Low
Description	When calculating fee shares in <code>requestWithdraw()</code> the share amount is rounded down → $feeShares = shares.mulDiv(withdrawFeeRate, PRECISION)$ Also, fee is rounded down in <code>_updateVaultAssets()</code> → $uint256 interestFee = totalInterest.mulDiv(feeRate, PRECISION)$
Recommendations	Round up fee calculation in favour of the protocol.
Comments / Resolution	Acknowledged.

Issue_06	Whitelist check can be bypassed
Severity	Informational
Description	The whitelist can be initialized with the <code>setWhitelist()</code> function and the <code>isInWhitelist()</code> function returns true if the whitelist array is currently empty. Therefore, a user can frontrun the <code>setWhitelist()</code> call (or any time before the whitelist is initialized) to <code>deposit()</code> and set a receiver of his choice.
Recommendations	The design choice should be acknowledged.
Comments / Resolution	Acknowledged.

Issue_07	<code>setWhitelist</code> does not emit an event
Severity	Informational
Description	Currently when calling <code>setWhitelist</code> to add or remove users to the whitelist, no event is emitted.
Recommendations	Consider emitting an event when calling <code>setWhitelist</code> .
Comments / Resolution	Fixed by following recommendations.

Issue_08	Allowing non zero values for both shares and amount lead to confusion
Severity	Informational
Description	Currently the deposit and withdrawal functions allow both shares and amount to be non zero. This can lead to confusion since if both values are non zero, only the amount value will be used and shares will be recalculated based on amount.
Recommendations	Consider requiring that only one value is > 0 , if both are > 0 revert.
Comments / Resolution	Acknowledged.

Issue_09	Users are not guarded against share loss when specifying assets
Severity	Informational
Description	There is no check that ensures that the user receives a minimum number of shares if they specify the assets to be deposited. Although the share price is set to 1 initially and share inflation isn't practically possible, it could still theoretically happen causing users to lose assets
Recommendations	Acknowledge/ensure protocol keeps an open position so that share inflation doesn't happen at all.
Comments / Resolution	Fixed by requiring shares minted is greater than 0.

Issue_10	Whitelist is not initialized during creation
Severity	Informational
Description	The whitelist will allow any user to deposit in case it is empty and the whitelist is not populated during initialization (only considering the contract's <code>initialize</code> function). This will allow any user to deposit into the contract before the first entry to the whitelist is pushed
Recommendations	Ensure whitelist is populated initially itself (either inside the <code>initialize</code> function or externally).
Comments / Resolution	Acknowledged.

Issue_11	Swap loss rate from USDC ↔ USD1 is not realised in the share price
Severity	Informational
Description	Consider the following scenario → 1.) User deposits 100 USD1 and it gets transferred to adapter contract , user receives 100 shares and <code>userTotalAssets</code> is 100. 2.) There we swap from USD1 to USDC , say we received 98 USDC after accounting for <code>toUSDCLossRate</code> (realistic loss rate would be much lower, using 2% for simplicity and explanation). 3.) This 98 USDC is deposited in Centrifuge. 4.) User requested a withdraw for 100 shares and amount comes out to be 100 (<code>totalAssets</code> is 100 and so is <code>totalSupply</code>, assume <code>withdrawFeeRate</code> is 0) 5.) <code>requestRedeem()</code> is performed for 98 USDC and on withdraw completion adapter has 98 USDC. 6.) This USDC will need to be swapped back to USD1 and after loss rate accounting say we have 96 USD1. 7.) Therefore, withdraw is queued for 100 USD1 in earn pool and the adapter has 96 USD1, only way to settle the batchId is when admin manually transfers a 4 USD1 surplus to adapter and then calls <code>finishEarnPoolWithdraw()</code>
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged, withdrawal fees will be used to cover the loss.

RWAAdapter

The **RWAAdapter** contract facilitates integration with the async vault. The **RWAAdapter** contract exposes functions for convenient conversion between USDC and USD1. These functions are needed since the **RWAEarnPool** asset is USD1 while the async vault underlying asset is USDC.

Due to the nature of the async vault, the **RWAAdapter** contract exposes 2 functions related to deposits, as well as 2 functions related to withdrawals.

Appendix: Async Vault Deposit

In order to initiate a deposit to the async vault, a call to **requestDeposit** must be made. At this point USDC has been transferred to the vault but no shares have yet been minted. In this stage the deposit is in a pending state waiting for confirmation from the HUB. Once confirmed, in order to receive vault shares, the contract calls on the mint function which in turn mints the shares to the contract.

Appendix Async Vault Withdraw

The withdrawal flow is a mirror of the deposit flow in the sense that first in order to initiate a withdrawal, a call to **requestRedeem** must be made. At this point shares have been transferred from the contract but assets have not yet been transferred to the contract. Once the pending stage has passed, the contract must then call on redeem or withdraw in order to withdraw the assets to the contract.

Assets may be swapped by sending tokens to the **OTCManager** which in turns sends those assets to the OTC wallet for swaps.

Core Invariants:

INV 1: Deposits and withdrawals must be queued first and confirmed due to the asynchronous nature of the vault.

INV 2: Swapping to and from USDC and USD1 is handled by the OTC manager and wallet

INV 3: Any interest generated will be added as rewards to the earn pool

Privileged Functions

- requestDepositToVault
- depositToVault
- depositRewards
- requestWithdrawFromVault
- withdrawFromVault
- finishEarnPoolWithdraw
- swapToken
- updateVaultAssets
- setFeeReceiver
- setFeeRate
- setToUSDCLossRate
- setToUSD1LossRate
- emergencyWithdraw
- grantRole
- revokeRole
- upgradeTo
- upgradeToAndCall

Issue_12	RWAAdapter is not compatible with handling losses
Severity	Medium
Description	When vault assets accounting is updated through <code>_updateVaultAssets()</code> → <pre>function _updateVaultAssets() private { uint256 newVaultTotalAssets = getVaultTotalAssets(); uint256 totalInterest = newVaultTotalAssets - lastVaultTotalAssets; // charge fee, asset is USDC uint256 interestFee = totalInterest.mulDiv(feeRate, PRECISION); fee += interestFee;</pre> This logic captures the profit correctly but during situations where the vault incurred a loss (vault share price decreases) the function would simply revert since <code>newVaultTotalAssets < lastVaultTotalAssets</code>. The entire logic of the adapter and integration with Centrifuge would break due to this since no new deposits and redemptions could be requested.
Recommendations	Losses should be notified similar to notifying profits to the <code>RWAEarnPool</code> .
Comments / Resolution	Fixed, <code>_updateVaultAssets</code> no longer reverts in the case where there is a loss. However, losses are still not passed to users.

Issue_13	No functionality to cancel a deposit
Severity	Low
Description	When a deposit is requested on Centrifuge's AsyncVault asset(USDC) is transferred from the <code>RWAAdapter</code> to the AsyncVault. Until the deposit request is approved vault shares can not be minted by the <code>RWAAdapter</code>. In case the deposit does not get approved AsyncVault provides a <code>cancelDepositRequest()</code> function to transfer back the assets transferred when the deposit was requested. But this function implementation is not present in the <code>RWAAdapter</code>, therefore if the deposit request is stuck and not approved for some reason the USDC deposited is also stuck in the AsyncVault.
Recommendations	Consider implementing the <code>cancelDepositRequest</code> functionality for such cases.
Comments / Resolution	Resolved. However, after discussions, it is recommended to remove the cancel functionality altogether i.e. <code>claimCancelDepositRequest</code> and <code>claimCancelRedeemRequest</code>, since this specific feature seems to be currently not existing (we still recommend the client to research this topic). Also it is to be noted that in case the functionality to cancel redeem requests is added in future, the shares obtained after <code>claimCancelRedeemRequest</code> will be recorded as interest rather than re-claimed shares which is flawed. We recommend conducting further research for that scenario when/how that could happen in the future.

Issue_14	using <code>redeem</code> and <code>maxRedeem</code> in withdrawals can lead to leaving dust amounts
Severity	Low
Description	When finalizing a withdrawal, it is recommended to use <code>withdraw</code> and <code>maxWithdraw</code> instead of <code>redeem</code> as the use of <code>redeem</code> and <code>maxRedeem</code> will lead to dust being left due to rounding when using <code>redeem</code>. This can be observed by dev comments: https://github.com/centrifuge/protocol/blob/25f38916503f881f404eebcb83de0b4e39cbd427/src/vaults/BaseVaults.sol#L301
Recommendations	Consider replacing <code>redeem</code> and <code>maxRedeem</code> with <code>withdraw</code> and <code>maxWithdraw</code> .
Comments / Resolution	Acknowledged.

Issue_15	Lack of event emission
Severity	Informational
Description	Currently many functions like <code>setFeeReceiver</code> which change state variables don't emit events. Emitting events on important updates allows offchain entities to process these updates in an efficient way
Recommendations	Consider emitting events inside such functions.
Comments / Resolution	Fixed by following recommendations.

Issue_16	Forcing to-be-redeemed shares to be calculated from asset amount can disallow withdrawing all the shares
Severity	Informational
Description	The amount of shares to be redeemed cannot be specified directly but is computed from passed in <code>amountUSDC</code> parameter by using the <code>convertToShares</code> function <pre> function requestWithdrawFromVault(uint256 amountUSDC) external onlyRole[BOT] { require(amountUSDC > 0, "Amount must be greater than zero"); // update vault assets and charge fee _updateVaultAssets(); // calculate shares to redeem uint256 redeemShares = IAsyncVault[vault].convertToShares(amountUSDC); </pre> But this doesn't always allow redeem the entire shares since there can be values for which the inverse doesn't exist
Recommendations	Adding a variant to specify the shares directly can bypass this issue.
Comments / Resolution	Acknowledged.

Issue_17	<code>setFeeRate</code> does not enforce max of 10%
Severity	Informational
Description	According to code comments on the function <code>setFeeRate</code> , the max <code>_feeRate</code> should be 10%. This is never checked and thus a fee rate more than 10% is possible.
Recommendations	Consider requiring that the free rate does not exceed 10%.
Comments / Resolution	Fixed by following recommendations.

OTCManager

The **OTCManager** is a very simple contract which is mainly used for sending tokens from the **RWAAdapter** to OTC wallet and from the **OTCManager** to the adapter. The function also includes emergency withdrawal capabilities in order to withdraw tokens in an emergency.

Core Invariants:

INV 1: Only USDC and USD1 can be transferred unless there is an emergencyWithdraw

INV 2: transfer can only occur to the OTC wallet and to the adapter, not including emergency withdrawals

Privileged Functions

- swapToken
- transferToAdapter
- grantRole
- revokeRole
- upgradeTo
- upgradeToAndCall

Issue_18	<code>_bot</code> is not validated for 0 address in <code>initialize</code>
Severity	Informational
Description	Although the other addresses are sanitized for 0 address, the <code>_bot</code> parameter is not validated <pre> function initialize(address_admin, address_manager, address_bot, address_adapter, address_otcWallet) external initializer { require[_admin != address[0], "Admin is zero address"]; require[_manager != address[0], "Manager is zero address"]; require[_adapter != address[0], "Adapter is zero address"]; require[_otcWallet != address[0], "OTC wallet is zero address"]; </pre>
Recommendations	Ensure <code>_bot</code> is not <code>address[0]</code> .
Comments / Resolution	Fixed by following recommendations.